

ECE-3103

Procedures

Md. Ebtidaul Karim

Introduction

- In writing programs often we need to use a particular sequence of instructions at several different points in a program.
- To avoid writing the sequence of instructions in the program each time we need them, we can write the sequence as a separate subprogram called 'procedure'.
- Each time we need to execute sequence of instructions contained in the procedure, we use the **CALL** instruction to send the 8086 to the starting address of the procedure in memory from the mainline program.
- A RET instruction at the end of the procedure returns execution to the next instruction in the mainline.

Single and Nested Procedure

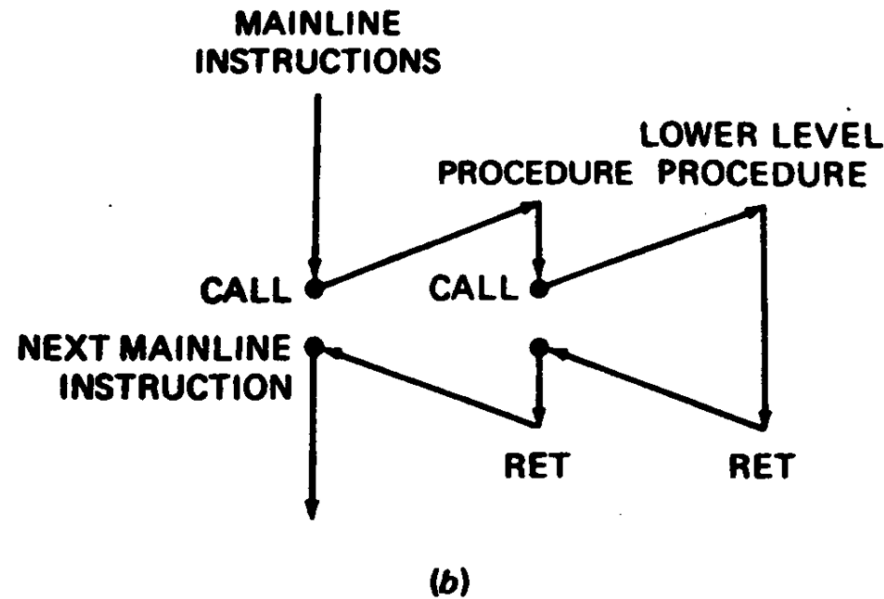
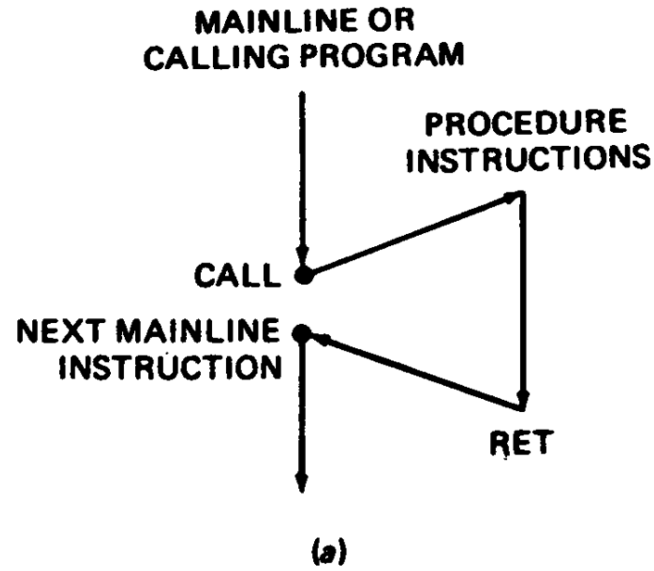


FIGURE 5-4 Program flow to and from procedures. (a) Single procedures. (b) Nested procedures.

- ✓ Procedures can even be nested.
- ✓ This means that one procedure calls another procedure as part of its instruction sequence.

8086 CALL and RET Instructions

- CALL instruction in the mainline program loads the IP and in some cases CS with the starting address of the procedure.
- The next instruction fetched will be the first address of the procedure.
- At the end of the procedure, a RET instruction sends execution back to the next instruction after the CALL in the mainline program.
- The RET instruction does this by loading the IP and, if necessary CS with the address of the next instruction after the CALL instruction.
- How does the RET instruction know where to return execution to?
- When a CALL instruction executes, it automatically stores the return address in stack.

CALL Instruction Overview

- CALL instruction performs two operations when it executes.
 - ❑ **First** it stores the address of the instruction after the CALL instruction on the stack.
- This address is called the return address because it is the address that execution will return to after the procedure executes.
- If the CALL is to a procedure in the same code segment, then the call is **near**, and only the IP contents will be saved on the stack.
- If the CALL is to a procedure in another code segment, then the call is **far**, and both the IP and CS contents will be saved on the stack.

CALL Instruction Overview

- ❑ The **second** operation of the CALL instruction is to change the contents of the IP and in some cases the content of the CS to contain the starting address of the procedure.
- CALL instruction is of four types. Such as:
 - ✓ Direct Within-Segment Near CALL.
 - ✓ Indirect Within-Segment Near CALL.
 - ✓ Direct Intersegment Far CALL.
 - ✓ Indirect Intersegment Far CALL.

Direct Within-Segment Near CALL

- It tells 8086 to produce the starting address of the procedure by adding a 16-bit signed displacement contained in the instruction to the contents of the IP.
- Starting address of the procedure can be anywhere in the range of -32,768 bytes to +32,767 bytes (2^{16} or 16 – *bit signed displacement*) from the address of the instruction after the CALL.
- If the procedure is in memory before the CALL instruction, then the displacement will be negative. It will be in 2's complement sign and magnitude form.

Indirect Within-Segment Near CALL.

- When this form of CALL executes, the IP is replaced with a 16-bit value from a specified register or memory location.
- As indicated by the coding template, the source of the value can be any of the eight 16-bit registers or a memory location specified by any one of the 24 addressing modes.
- The instruction CALL BP, for example, will do a near call to the offset contained in BP.
- In other words, the value in BP will be put in the instruction pointer.

Direct Intersegment Far CALL.

- Here as the procedure is in another segment, we have to change both the IP and the CS.
- In this form of CALL instruction, the new value for the IP is written in as bytes 2 and 3 of the instruction code.
- The new value for the CS is written in as bytes 4 and 5 of the instruction code.
- In both cases, low byte is written before the high byte.

Indirect Intersegment Far CALL

- This form of CALL instruction replaces the IP and the CS contents with two 16-bit values from memory.
- Since two 16-bit values are needed, the values cannot come from a register.
- The MOD, R/M byte in the instruction is used to specify the addressing modes for the memory location where the 8086 goes to get the new value.
- The first word from memory is put in the IP and the second word from memory is put in the CS.
- For example the instruction `CALL DWORD PTR[BX]` will get a new value for IP from `[BX]` and `[BX+1]` in the data segment and new value for CS from offset `[BX+2]` and `[BX+3]`.

CALL Instruction Overview

CALL = Call

Within segment or group, IP relative

Opcode	DispLow	DispHigh
--------	---------	----------

Opcode	Clocks	Operation
E8	19	$IP \leftarrow IP + \text{Disp16} - (SP) \leftarrow \text{return link}$

Within segment or group, Indirect

Opcode	mod 010 r/m					
--------	-------------	--	--	--	--	--

Opcode	Clocks	Operation
FF	16	$IP \leftarrow \text{Reg16} - (SP) \leftarrow \text{return link}$
FF	21 + EA	$IP \leftarrow \text{Mem16} - (SP) \leftarrow \text{return link}$

CALL 2050H → Direct Call

CALL [BX] → Indirect Call

Inter-segment or group, Direct

Opcode	offset-low	offset-high	seg-low	seg-high
--------	------------	-------------	---------	----------

Opcode	Clocks	Operation
9A	28	$CS \leftarrow \text{segbase}$ $IP \leftarrow \text{offset}$

Inter-segment or group, Indirect

Opcode	mod 011 r/m	mem-low	mem-high
--------	-------------	---------	----------

Opcode	Clocks	Operation
FF	37 + EA	$CS \leftarrow \text{segbase}$ $IP \leftarrow \text{offset}$

8086 RET Instruction

- When the 8086 does a near call, it saves the IP value for the instruction after the CALL on the stack.
- A RET at the end of the procedure copies this value from the stack back to the IP to return execution to the mainline program.
- When the 8086 does a far call, it saves the contents of both IP and CS on the stack.
- A RET at the end of the procedure copies this value from the stack back to the IP and CS register to return execution to the mainline program.

8086 RET Instruction

- 8086 has four forms of RET instruction.
- Such as:
 - ✓ Within segment form of RET
 - Normal form
 - SP form
 - ✓ Inter-segment form of RET
 - Normal form
 - SP form

8086 RET Instruction

- The simple **within-segment** form of RET copies a word from the top of the stack to the IP.
- The **within-segment adding immediate to SP** form is also used to return from a near procedure.
- It will copy the word at the top of the stack to the IP and also add an immediate number contained in the instruction to the contents of the SP.
- The **intersegment form** of the RET instruction is used to return from far procedure.

8086 RET Instruction

- When this form of RET instruction executes, it will copy the word from the top of the stack to the IP.
- It will then increment the SP by 2 and copy the next word from the top of the stack to the CS register.
- The intersegment adding immediate to SP form of the instruction also copies a new value for IP and a new value for CS from the stack.
- However, it also adds immediate number contained in the instruction code to SP.

8086 RET Instruction

RET = Return from Subroutine

Opcode

Opcode	Clocks	Operation
C3	8	intra-segment return
CB	18	inter-segment return

Return and add constant to SP

Opcode	DataL	DataH
--------	-------	-------

Opcode	Clocks	Operation
C2	12	intra-segment ret and add
CA	17	inter-segment ret and add

The 8086 Stack

- Stack is an entire 64-Kbyte segment of a memory. It has 3 function.
 1. The stack is a section of memory we set aside for storing return addresses.
 2. The stack is also used to save the contents of registers for the calling program while a procedure executes.
 3. Stack is also used to hold data or addresses that will be acted upon by a procedure.
- 8086 contains a SS and SP register. SS register is used to hold the upper 16 bits of the starting address we give to the stack segment.

The 8086 Stack

- SP register is used to hold the offset of the last word written on the stack.
- Physical Address of the stack = $SS \times 10 + SP$.
- At the start of the program we must initialize the SP register to point to the top of the memory we set aside as a stack.
- SP register is automatically decremented by 2 before a word is written to the stack.
- This is known as **Push** operation.

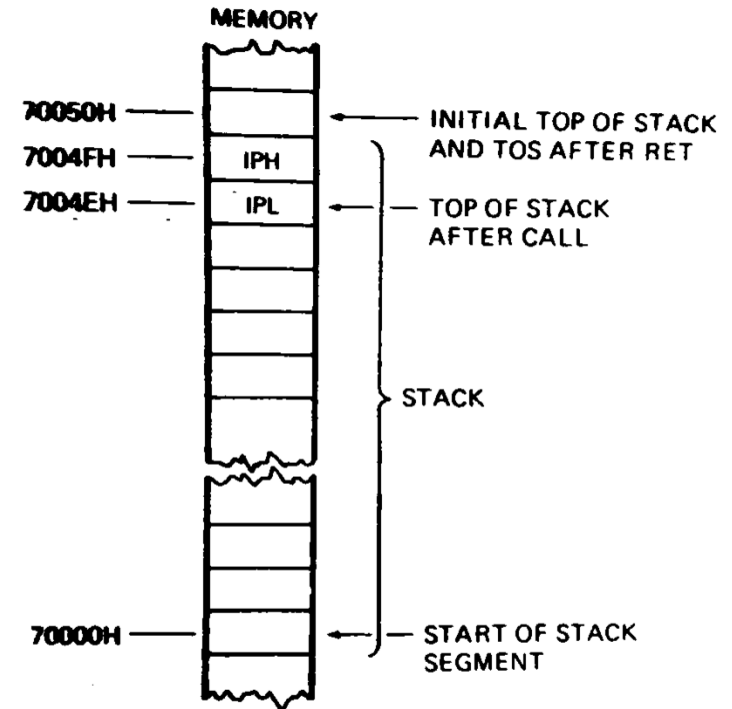


FIGURE 5-7 Stack diagram showing how the return address is pushed onto the stack by CALL.

The 8086 Stack

- When a near RET instruction executes, the IP value stored in the stack will be copied back to the IP register, and the SP register will be automatically incremented by 2.
- This is known as **Pop** operation.
- After a CALL-RET sequence, then, the SP register is again pointing to the Initial top-of-stack location.

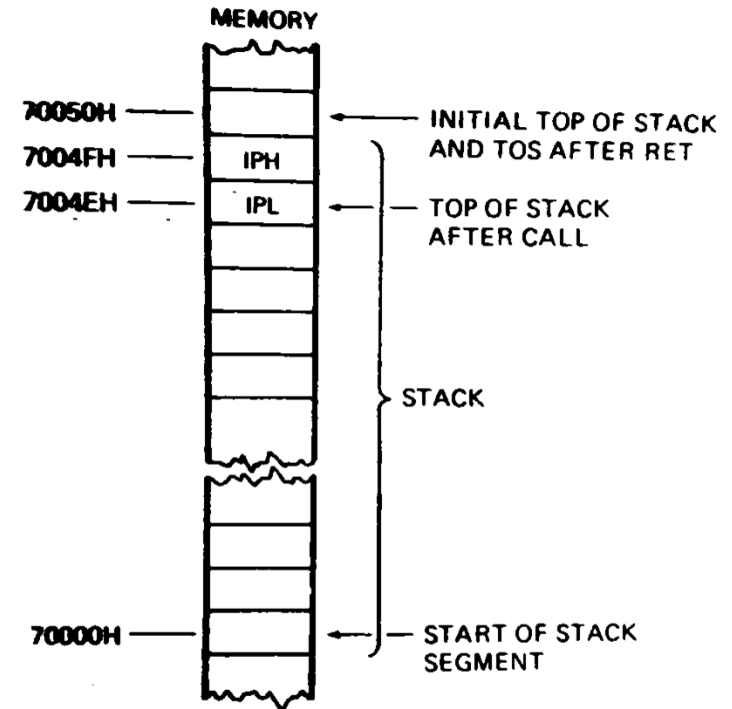
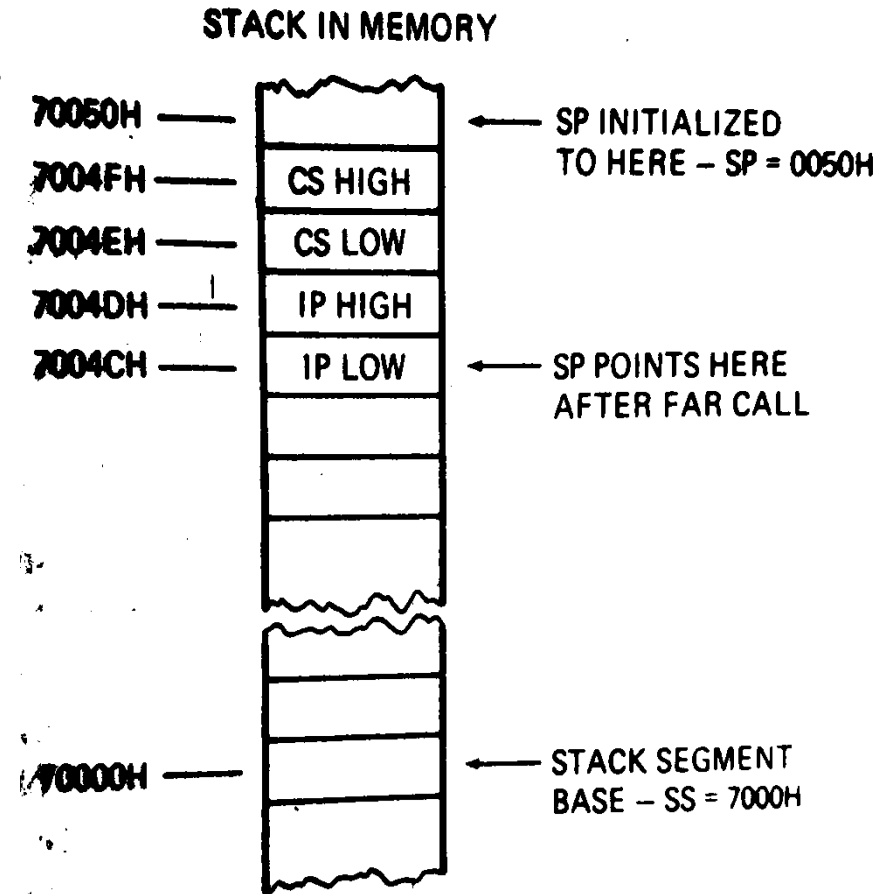


FIGURE 5-7 Stack diagram showing how the return address is pushed onto the stack by CALL.

The 8086 Stack- Far CALL

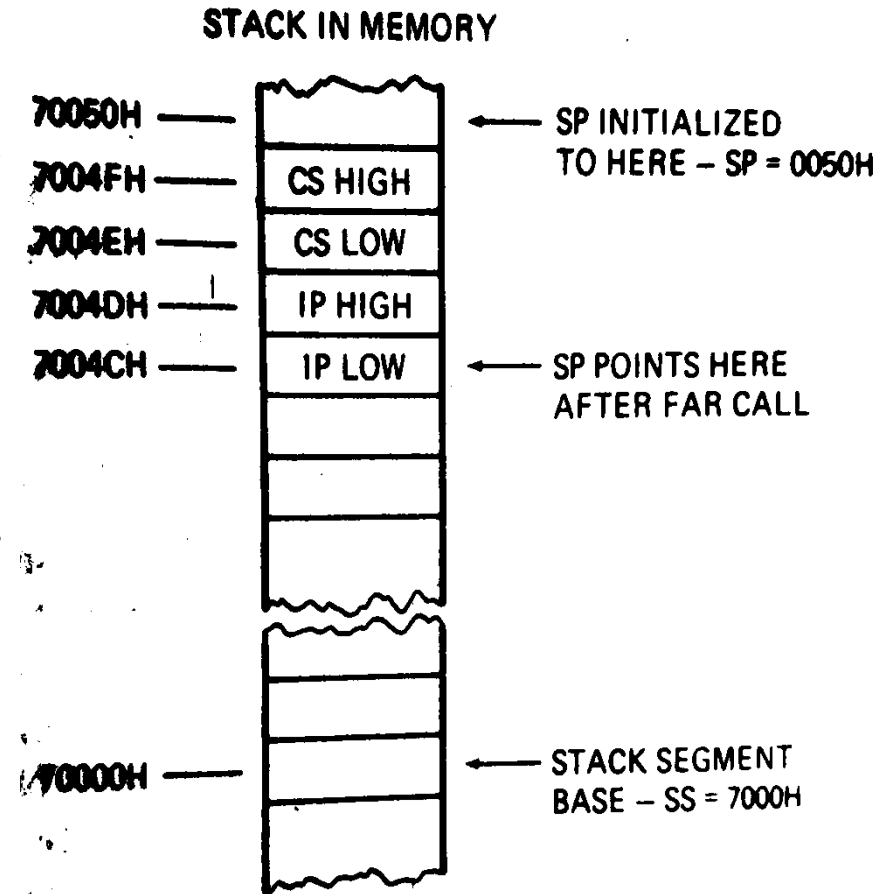
- When the 8086 executes a far CALL instruction, it decrements the SP by 2 and copies the content of the CS register to the stack.
- It then decrements the SP by 2 again and copies the offset of the next mainline instruction from the instruction pointer to the stack.
- Both are **Push** operation.
- After a far CALL the top of the stack will be four addresses lower than it was before far call.



(b)

The 8086 Stack- Far CALL

- A far RET used at the end of a far procedure will copy the word from the top of the stack to the IP and increment the SP by 2.
- It will then copy the word from the new top of the stack to the CS register.
- The next instruction will then be fetched from the physical address after the far CALL instruction.
- The top of the stack will be back to where it was before the CALL and RET.
- This operation is known as **Pop** operation.



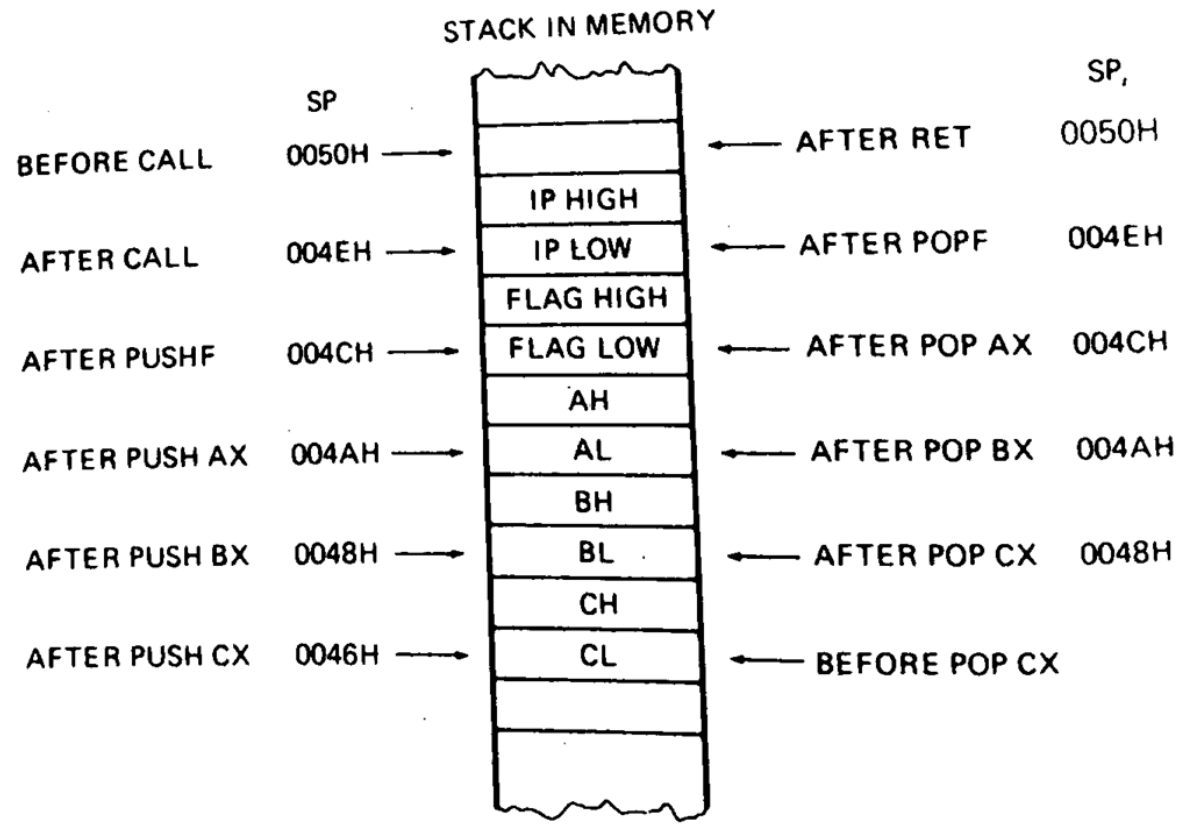
(b)

PUSH and POP Instruction

□ As stack operates in LIFO manner we have to POP off in reverse order.

```
MOV SP, 0050H
CALL MULTO
MULTO PROC NEAR
    PUSHF
    PUSH AX
    PUSH BX
    PUSH CX
    :
    POP  CX
    POP  BX
    POP  AX
    POPF
    RET
MULTO ENDP
```

(a)



(b)

Self Study(For Better Understanding of Programming)

- **Segments and ENDS Directives**(These are used in the same way that parenthesis are used to group like terms in algebra).
- A group of data segments or a group of instruction statements contained between SEGMENT and ENDS directives is called a logical segment.
- Naming Data and Addresses:

EQE DB DW DD

Self Study(For Better Understanding of Programming)

- Program works with three general categories of data:

1. Constant
2. Variable
3. Address

❑ **EQU Directive:** CORRECTION_FACTOR EQU 07H in a program such as our previous example, would tell the assembler to insert the value 07H every time it finds the name CORRECTION_FACTOR in a program statement.

❑ **DB Directives:** DB directive after a name specifies that the data is of type byte.

Self Study(For Better Understanding of Programming)

- **DW and DD Directive:** It is used to specify that the data is of type word (16 bits) and DD is used to specify that the data is of type double-word (32 bits).
- ✓ For example: `PRODUCT DW 2 DUP(0)` sets aside storage of two words in memory and gives the starting address of the first word the name `PRODUCT`.
- ✓ The `DUP(0)` part of the statement tells the assembler to initialize the two words to all zeros.
- **Types of Numbers Used in Data Statements.**

Self Study(For Better Understanding of Programming)

❑ Assembly language source program to multiply two 16 bit binary numbers to give a 32 bit result.

❑ The ASSUME Directive

❑ Initializing Segment Register

❑ The END Directives.

```
; 8086 PROGRAM F3-14.ASM
;ABSTRACT : This program multiplies the two 16-bit words in the memory
; locations called MULTIPLICAND and MULTIPLIER. The result
; is stored in the memory location, PRODUCT
;REGISTERS : Uses CS, DS, AX, DX
;PORTS : None used

DATA_HERE SEGMENT
MULTIPLICAND DW 204AH ; First word here
MULTIPLIER DW 3B2AH ; Second word here
PRODUCT DW 2 DUP(0) ; Result of multiplication here
DATA_HERE ENDS

CODE_HERE SEGMENT
ASSUME CS:CODE_HERE, DS:DATA_HERE
START: MOV AX, DATA_HERE ; Initialize DS register
MOV DS, AX
MOV AX, MULTIPLICAND ; Get one word
MUL MULTIPLIER ; Multiply by second word
MOV PRODUCT, AX ; Store low word of result
MOV PRODUCT+2, DX ; Store high word of result
INT 3 ; Wait for command from user
CODE_HERE ENDS
END START
```

Self Study(For Better Understanding of Programming)

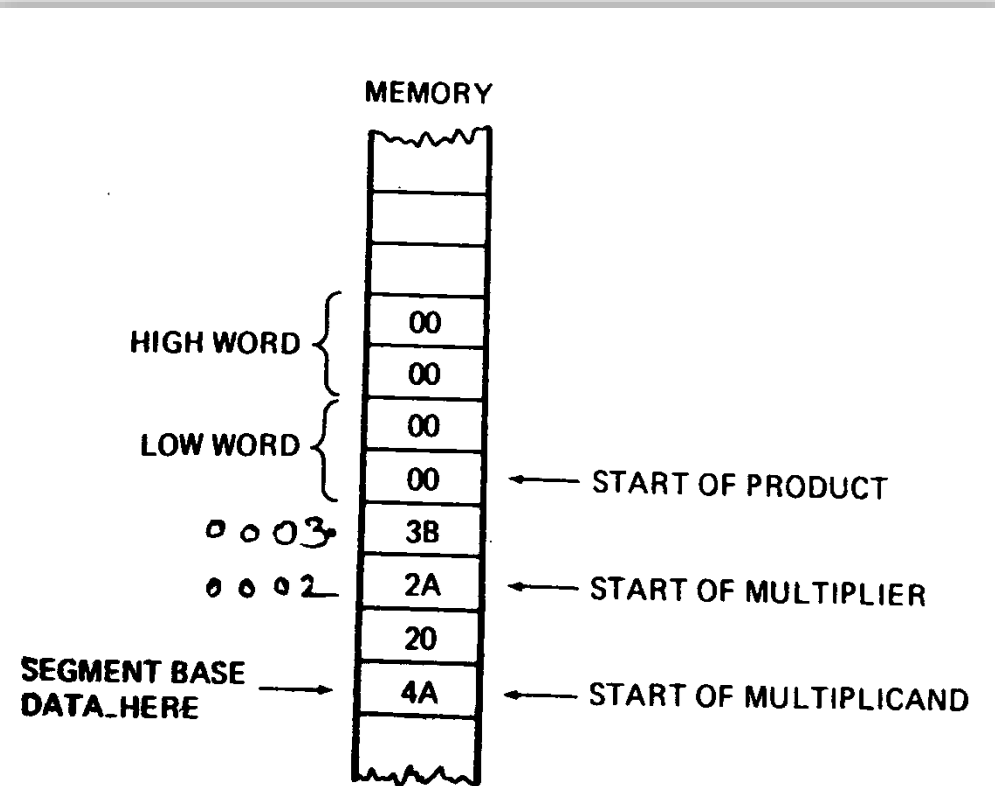
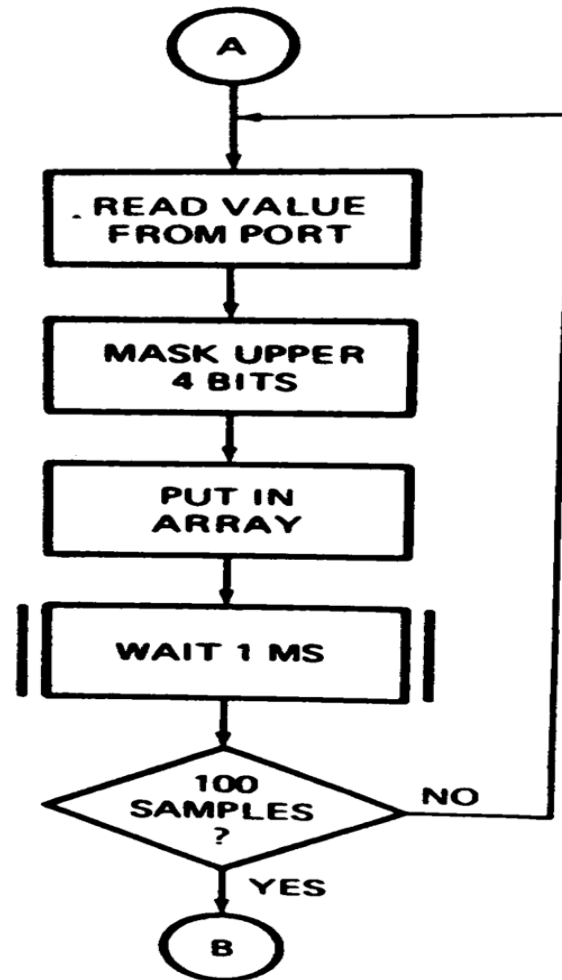


FIGURE 3-15 Data arrangement in memory for multiply program.

Self Study(For Better Understanding of Programming)

Read 100 samples of data at 1-ms interval



PRESSURE_PORT EQU 0FFF8H

DATA SEGMENT
PRESSURES DW 100 DUP(0) ; Set up array of 100 words
NBR_OF_SAMPLES EQU (($\$$ -PRESSURES)/2)
DATA ENDS

STACK_SEG SEGMENT
DW 40 DUP(0) ; set stack length of 40 words
STACK_TOP LABEL WORD
STACK_SEG ENDS

CODE SEGMENT
ASSUME CS:CODE, DS:DATA, SS:STACK_SEG
START: MOV AX, DATA ; Initialize data segment register
MOV DS, AX
MOV AX, STACK_SEG ; Initialize stack segment register
MOV SS, AX
MOV SP, OFFSET STACK_TOP ; Intialize stack pointer to top of stack

LEA SI, PRESSURES ; Point SI to start of array
MOV BX, NBR_OF_SAMPLES ; Load BX with number of samples
MOV DX, PRESSURE_PORT ; Point DX at input port
NEXT_VALUE: IN AX, DX ; Read data from port
AND AX, 0FFFH ; Mask upper 4 bits
MOV [SI], AX ; Store data word in array
✓ CALL WAIT_1MS ; Delay 1 ms
INC SI ; Point SI at next location in array
INC SI
DEC BX ; Decrement sample counter
JNZ NEXT_VALUE ; Repeat until 100 samples done
STOP: NOP

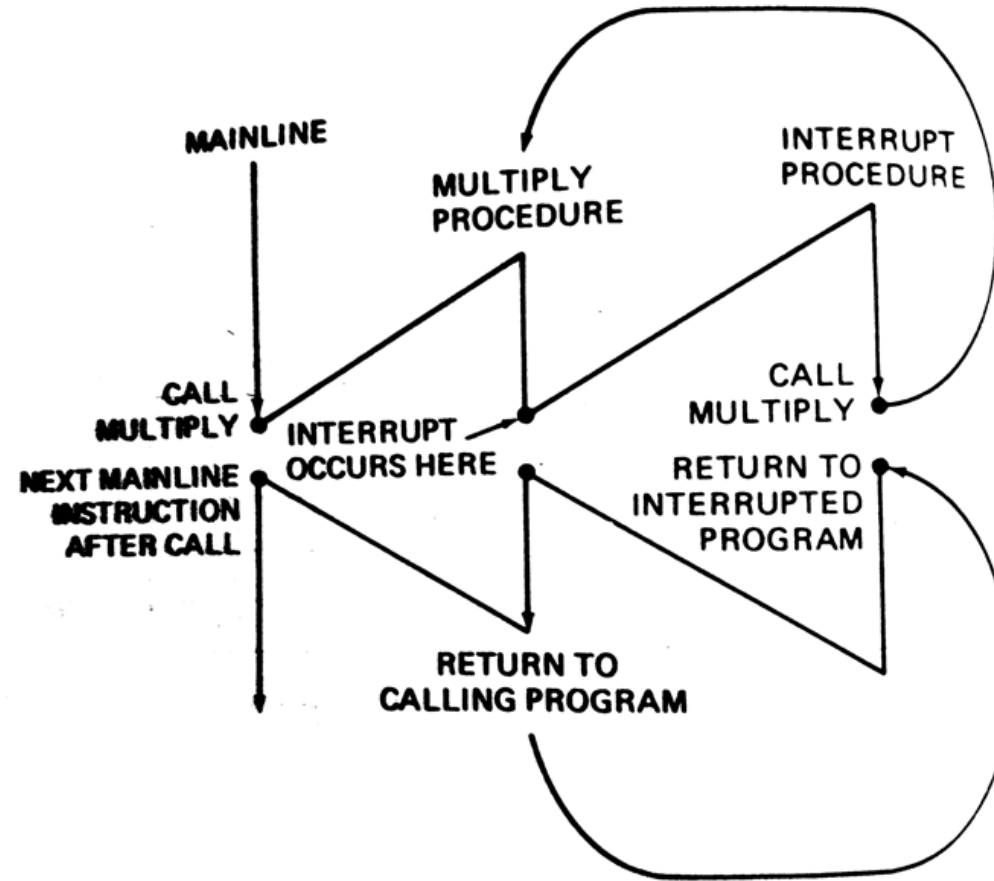
WAIT_1MS PROC NEAR
MOV CX, 23F2H ; Load delay constant into CX
HERE: LOOP HERE ; Loop until CX = 0
RET
WAIT_1MS ENDP

CODE ENDS
END
END

Self Study(For Better Understanding of Programming)

- Fig 5.14, 5.15, 5.16, 5.17, 5.18, 5.19
- Passing parameters to and from procedures.

Reentrant Procedure



Reentrant Procedure

- What is interrupt service procedure??
- Let a temperature sensor in a flow-solder machine could be connected to the interrupt input.
- If the temperature gets too high, the sensor sends an interrupting signal to the 8086.
- 8086 will then stop whatever it is doing and go to a procedure which takes whatever steps are necessary to cool down the solder batch.
- This procedure is called an **interrupt service procedure(ISP)**.

Reentrant Procedure

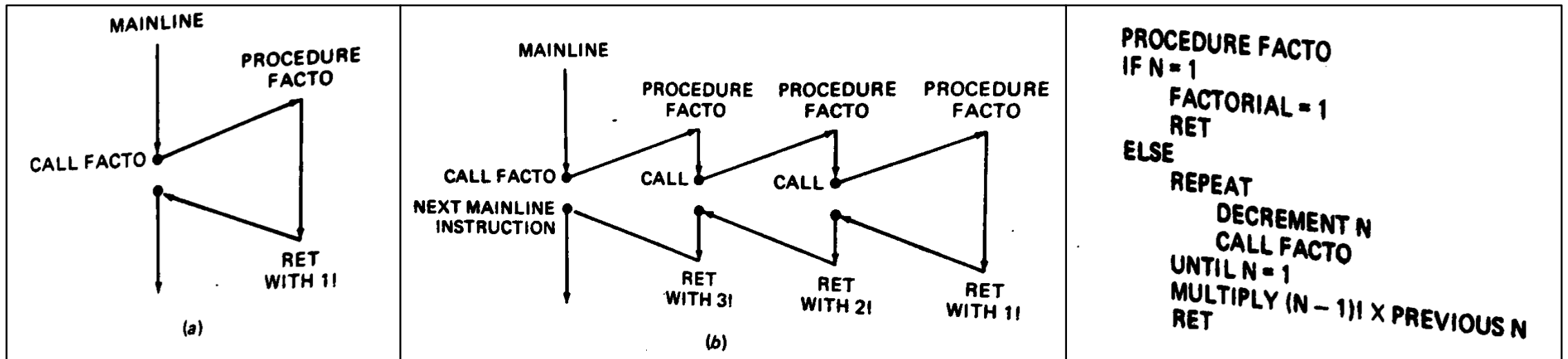
- Here when the interrupt occurs, execution goes to the ISP.
- The ISP then calls the multiply procedure when it needs it.
- The RET instruction at the end of the multiply procedure returns execution to the ISP.
- A special return instruction at the end of the ISP returns execution to the multiply procedure where it was executing when the interrupt occurred.
- Multiply procedure must be written in such a way that it can be interrupted, used and reentered without losing or writing over anything.

Reentrant Procedure

- A procedure which can function in this way is said to be **reentrant**.
- To be reentrant, a procedure must first of all push the flags and all the registers used in the procedure.
- Also to be reentrant, a program should use only registers or the stack to pass parameters.

Recursive Procedure

- Example: Factorial



Recursive Procedure

- A recursive procedure is a procedures which call itself.
- ✓ **If** $N = 1$ **THEN** factorial = 1
- ✓ **ELSE** factorial = $N * (\text{factorial of } N-1)$
- If the value of N passed to the procedure is 1, then the procedure simply loads 1 into the stack location reserved for N!
- If we call the procedure with $N=3$, the procedure will call itself to compute $(N-1)!$ Or $2!$.
- It will then call itself to compute the value of the next $(N-1)!$ Or $1!$
- Since $1!=1$, the procedure will return this value to the program that called it.

Recursive Procedure

- In this case the program that called it was a previous execution of the same procedure that needed this value to compute $2!$.
- Given this value, it will compute $2!$ And return the value to the program that called it.
- Here again, the program that called it was a previous execution of the same procedure that needed $2!$ To compute the factorial of 3.
- Given the factorial of 2, this call of the procedure can now compute $3!$ And return to the program that called it.
- For the example, return will now be to the mainline program.

Thank You

Thank You